

Data Processing Pipeline

Gas Injury Detection — MEMS and SPEC Sensors

Experiment Data

April 2026

Contents

1 Overview

Two independent processing pipelines clean the raw CSV recordings before any modelling. Each pipeline is implemented in a Jupyter notebook and exports `.pkl` files to the `processed/` directory.

Notebook	Sensors	Output files
<code>mems_filter.ipynb</code>	VOC, NH3, HCHO (metal-oxide)	<code>mems_*.pkl</code>
<code>spec_filter.ipynb</code>	H2S, EtOH (electrochemical)	<code>spec_*.pkl</code>

The datasets processed are:

Key	Source file	Session	Notes
SWEAT_1a	20260405-experiment/sweat.csv	Apr 05	Up to 4000 s
SWEAT_1b	20260405-experiment/sweat.csv	Apr 05	4000 s onward; elapsed reset
BLOOD_0	20260406-experiment/1.5_blood.csv	Apr 06	Full session
BLOOD_1	20260406-experiment/1.5_blood.2.csv	Apr 06	Trimmed: $t \geq 1750$ s

Sweat 0 (20260404-experiment/sweat.csv) is excluded from training — the NH3 pin was disconnected (miswired) for that session — but it is used as an out-of-sample validation target.

2 MEMS Pipeline (`mems_filter.ipynb`)

2.1 Data Loading

Raw CSV is read with `wall_time` parsed as a timestamp. Elapsed time is computed as:

$$t_{\text{elapsed}} = \text{wall_time} - \text{wall_time}[0] \quad [\text{seconds}]$$

An optional trim is applied: rows with $t < t_{\min}$ or $t \geq t_{\max}$ are discarded before any processing.

For Sweat 1, the session is split at $t_{\text{reset}} = 4000$ s (a recording restart event):

- **SWEAT_1a:** $t < 4000$ s — contains sweat samples 2 and 3 (introduced ≈ 3850 s).
- **SWEAT_1b:** $t \geq 4000$ s — elapsed reset to 0; contains sweat sample 1 (introduced ≈ 570 s of sub-session time).

2.2 Artifact Removal

Known electrical or handling artifacts are patched *before* smoothing by replacing the affected time range with `NaN` and interpolating linearly between the clean neighbours. Per-dataset artifact windows:

Dataset	Channel	t_{start} (s)	t_{end} (s)
SWEAT_1a	VOC, NH3, HCHO	2950	3100
SWEAT_1b	HCHO	540	570
BLOOD_0	NH3	1970	2030

BLOOD_0 NH3 also has isolated zero readings interpolated before artifact removal (sensor dropout artefacts).

2.3 Smoothing — Causal Exponential Moving Average

After artifact patching, each MEMS channel is smoothed with a causal Exponential Moving Average (EMA):

$$\hat{x}[n] = \alpha x[n] + (1 - \alpha) \hat{x}[n - 1], \quad \alpha = \frac{2}{\text{span} + 1}$$

Default span is 30 samples (≈ 30 s at 1 Hz). SWEAT_1b NH3 uses a larger span of 80 to suppress its higher noise level.

Why causal EMA: The model will run in real time on a microcontroller. Using a non-causal filter (e.g. Savitzky–Golay, zero-phase Butterworth) during training but a causal filter at inference would introduce a train/inference mismatch. Causal EMA is identical offline and online.

2.4 Baseline and Sensor Drift Correction

Metal-oxide sensors exhibit a slow resistive drift even in clean air. A linear trend is estimated from a *pre-sample baseline window* and subtracted from the full signal:

1. Select rows in the baseline window $[t_0, t_1]$ (before sample introduction).
2. Fit a degree-1 polynomial to the EMA-filtered signal in that window:

$$\hat{y}(t) = a \cdot t + b \quad \text{via least squares on } t \in [t_0, t_1]$$

3. Subtract the extrapolated trend from the entire session:

$$y_{\text{corr}}(t) = \max(y(t) - \hat{y}(t), 0)$$

The $\max(\cdot, 0)$ clip prevents negative readings.

Baseline windows per dataset:

Dataset	t_0 (s)	t_1 (s)
SWEAT_1a	1500	2800
SWEAT_1b	0	570
BLOOD_0	0	1900
BLOOD_1	1750	2440

2.5 Export

Drift-corrected channels [`elapsed_s`, `voc`, `nh3`, `hcho`, `temp_C`, `rh_pct`] are exported as `processed/mems_{}` for each dataset.

3 SPEC Pipeline (`spec_filter.ipynb`)

3.1 PPM Conversion

Raw voltage readings from the SPEC electrochemical sensors are converted to PPM using the transimpedance amplifier (TIA) gain and manufacturer sensitivity code:

$$m = S_{\text{code}} \times R_{\text{TIA}} \times 10^{-6} \quad [\text{V/ppm}]$$

$$\text{ppm} = \max\left(\frac{V_{\text{gas}} + \delta V - V_{\text{ref}}}{m}, 0\right) \times s$$

where δV is a per-sensor voltage offset and s is a multiplicative scale factor. Parameters used:

Sensor	S_{code} (nA/ppm)	R_{TIA} (k Ω)	m (V/ppm)	δV (V)	s
H2S	216.09	49.9	1.078×10^{-2}	0.000	1.5
EtOH	21.5	249.0	5.354×10^{-3}	+0.100	1.0

3.2 Outlier Detection and Handling

Two independent detectors are run on the raw PPM signal; a sample flagged by *either* is treated as an outlier.

Rolling IQR

$$\text{outlier if } x < Q_1 - k \cdot \text{IQR} \text{ or } x > Q_3 + k \cdot \text{IQR}$$

Computed over a causal rolling window of 60 samples with $k = 2.0$.

Rolling Rate-of-Change (ROC)

$$\text{outlier if } \frac{|\Delta x[n]| - \mu_{\text{ROC}}}{\sigma_{\text{ROC}}} > z_{\text{thresh}}$$

where μ_{ROC} and σ_{ROC} are the rolling mean and standard deviation of $|\Delta x|$ over 60 samples, and $z_{\text{thresh}} = 3.0$.

Flagged samples are replaced by linear interpolation between the surrounding clean values (forward/backward filled at boundaries).

3.3 Smoothing — Three-Stage Filter

The outlier-handled signal is smoothed with a three-stage pipeline:

1. **Savitzky–Golay (SG)** pre-smoother: window = 121 samples, polynomial order = 3. Removes rapid point noise while preserving peak shapes.
2. **Causal Butterworth low-pass**: 4th-order, $f_c = 0.005$ Hz at $f_s = 1$ Hz. Extracts the slow trend. Implemented with `lfilter` and matched initial conditions (`lfilter_zi`) to avoid edge transients.

Note: The SG stage is non-causal (used here for noise pre-processing only); the Butterworth stage is causal. At inference, the SG step may be replaced by a running EMA of equivalent span.

3.4 Boot-up Stabilization Detection

After power-on, SPEC sensors drift substantially before settling. Each sensor uses a dedicated algorithm to detect the stabilization point.

H2S — Dual Moving Average Convergence

A short-term MA (60 s) and long-term MA (300 s) are computed on the outlier-handled signal. The sensor is considered stable when:

$$|\text{MA}_{\text{short}}(t) - \text{MA}_{\text{long}}(t)| < \theta \quad \text{for } \geq 300 \text{ consecutive samples}$$

with $\theta = 0.05$ ppm. The first 1200 samples are always excluded (sensor is guaranteed unstable during warmup).

EtOH — Rolling Slope Threshold

EtOH decays exponentially from a high boot value; the dual-MA gap closes too slowly. Instead, the three-stage filtered signal is used:

$$\text{stable when } |\overline{\Delta x}|_w < \theta_{\text{slope}} \quad \text{for } \geq 300 \text{ consecutive samples}$$

where the rolling mean is computed over a long window and θ_{slope} is a dataset-specific threshold.

3.5 Stabilization Offset Correction

The signal value at the detected stabilization point is used as a per-boot baseline offset. All readings are shifted so the baseline is zero:

$$y_{\text{corr}}(t) = \max(y(t) - y(t_{\text{stable}}), 0)$$

If stabilization is not detected, the mean of the first 300 samples is used as a fallback.

3.6 Sweat 1 Session Split

The Sweat 1 session (20260405) contains two distinct sub-sessions separated by a recording restart at $t = 4000$ s. After offset correction, the session is split:

- **SPEC SWEAT_1a:** $t \leq 4000$ s
- **SPEC SWEAT_1b:** $t > 4000$ s, elapsed reset to 0

3.7 Export

Offset-corrected channels [elapsed_s, h2s_ppm, etoh_ppm] are exported as `processed/spec_{key}.pkl` for each dataset.

4 Merge and Feature Engineering (combined.ipynb)

4.1 Merge

MEMS and SPEC pickle files are inner-joined on `elapsed_s` to produce one DataFrame per dataset with all seven channels: `voc`, `nh3`, `hcho`, `h2s_ppm`, `etoh_ppm`, `temp_C`, `rh_pct`.

4.2 Labelling

Each row is assigned one of three class labels based on manually annotated time windows (from experimental README):

Label	Class	Dataset	Window (s)
0	Baseline	All	Rows outside sample windows
1	Sweat	SWEAT_1a	2870 – 3850
1	Sweat	SWEAT_1b	570 – 2330
2	Blood	BLOOD_0	1900 – 3730
2	Blood	BLOOD_1	2440 – 3800

4.3 Helper Columns

The following features are computed per row, causally (using only past data), for each channel in [voc, nh3, hcho, h2s_ppm, etoh_ppm, rh_pct]:

Column	Description
<code>_roc</code>	First difference $x[n] - x[n - 1]$
<code>_acc</code>	Second difference (diff of diff)
<code>_roll_mean_{w}</code>	Causal rolling mean over $w \in \{15, 30, 60\}$ samples
<code>_roll_std_{w}</code>	Causal rolling std over w samples
<code>_roll_roc_{w}</code>	Causal rolling mean of $ \text{ROC} $ over w samples

`temp_C` is excluded from feature columns; `rh_pct` contributes helper columns only (not its raw value).

4.4 Windowed Feature Extraction

Non-overlapping 60-second windows are slid separately over each *(session, label)* pair — windows never cross label boundaries. Within each window, four aggregates are computed per feature column:

$$\{\text{mean, std, max, slope}\}$$

where slope is the linear regression coefficient of the column against sample index within the window.

4.5 Class Balancing

Baseline windows greatly outnumber sample windows. The baseline set is randomly under-sampled to match the size of the larger sample class (sweat or blood), producing a balanced `windows_balanced` DataFrame.

5 Model — Random Forest

5.1 Run 1

- 200 decision trees, `class_weight="balanced"`
- All feature columns (raw aggregates + ROC + rolling mean/std/ROC + acceleration)
- Leave-One-Session-Out (LOSO) cross-validation via `GroupKFold(n_splits=4)`
- Mean validation accuracy: 0.752, std: 0.253
- Fold 3 (held-out: blood_0) collapsed to 0.333 — blood_0 EtOH max ≈ 14 ppm vs blood_1 max ≈ 0.3 ppm; model had only seen blood_1 for the blood class.

5.2 Run 2

- 100 decision trees, `class_weight="balanced"`
- Feature reduction: remove `_roll_mean_*`, `_roll_std_*`, `_acc` columns; keep raw aggregates and ROC columns only
- 80/20 stratified train/test split on `windows_balanced`
- Per-class precision and recall reported on the held-out 20%

Future sessions (new blood/sweat recordings) will serve as true out-of-sample validation.